Tenemos esta situación: un mi sistema operativo GNU/Linux, un entorno conda dentro del cual se desarrolla un Agente de IA (con python) que recibe instrucciones a través del sistema de mensajería Telegram. Por cuestiones de seguridad, las instrucciones en python que el Agente de IA recibe de Telegram, se ejecutan en un "**sandbox de Docker**". En dicho sandbox se instaló **FreeCAD** y **KiCAD** como apps de apoyo al Agente de IA. El agente de IA reconoce que debe ejecutar instrucciones de FreeCAD cuando por Telegram recibe por ejemplo el siguiente mensaje: "/freecad un cilindro de radio 10 y altura 30". El mensaje le llega al Agente de IA pero al tratar de ejecutarlo arroja un mensaje de **ERROR** indicando que no consigue el ejecutable FreeCAD. Y estoy complemente seguro de haberlo instalado, a continuación presento el script python **build_sandbox.py** que instala los paquetes:

```
#!/usr/bin/env python3
import docker
import sys
import os

def main():
   try:
       client = docker.from_env()
   except Exception as e:
       print(f"❌ Error conectando a Docker: {e}")
       sys.exit(1)

   print("🐳 Construyendo imagen de Sandbox personalizada
(agent-sandbox)...")
   print("   Esto instalará: pandas, numpy, matplotlib, requests,
beautifulsoup4, pypdf.")
   print("   ⏳ Paciencia, esto puede tardar unos minutos la primera
vez.")

   # Definir el contenido del Dockerfile
   dockerfile_content = """
FROM ubuntu:22.04
# Evitar archivos .pyc y buffering
ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
# Instalar dependencias del sistema y librerías de python en una sola capa
para optimizar el tamaño de la imagen
```

```
# Añadir PPA de KiCad para evitar conflictos de dependencias (libocct) con
FreeCAD
RUN apt-get update && apt-get install -y --no-install-recommends
software-properties-common gnupg \
    && add-apt-repository -y ppa:freecad-maintainers/freecad-stable \
    && add-apt-repository -y ppa:kicad/kicad-8.0-releases \
    && add-apt-repository -y universe \
    && apt-get update && apt-get install -y --no-install-recommends \
    python3 \
    python3-pip \
    python3-dev \
    build-essential \
    kicad \
    freecad \
    && pip3 install --no-cache-dir pandas numpy matplotlib requests
beautifulsoup4 pypdf opencv-python-headless skidl pathfinding \
    && apt-get purge -y --auto-remove python3-dev build-essential \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
# Asegurar que 'python' apunte a python3 para compatibilidad
RUN ln -s /usr/bin/python3 /usr/bin/python
WORKDIR /app
"""

   # Crear un archivo temporal para el contexto de build
   project_root =
os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
   dockerfile_path = os.path.join(project_root, "Dockerfile.sandbox")

   with open(dockerfile_path, "w") as f:
       f.write(dockerfile_content)

   try:
       # Construir la imagen
       image, logs = client.images.build(path=project_root,
dockerfile="Dockerfile.sandbox", tag="agent-sandbox:latest", rm=True,
nocache=True)
```

```
        print("\n✅ Imagen 'agent-sandbox:latest' construida
exitosamente.")
        print("   ✅ Soporte para KiCad (pcbnew), FreeCAD y Electrónica
incluido.")
        print("   Ahora tus scripts de Python volarán. 🚀")
    except docker.errors.BuildError as e:
        print(f"\n❌ Error en el build: {e}")
        for line in e.build_log:
            if 'stream' in line: print(line['stream'].strip())
    except Exception as e:
        print(f"\n❌ Error inesperado: {e}")

if __name__ == "__main__":
    main()
```

Sin embargo la ejecución de KiCAD y no da problemas. Pero con FreeCAD si da problemas.

Análisis del posible problema:

Se Tiene una arquitectura robusta (Conda + Docker + Telegram), y el hecho de que KiCAD funcione pero FreeCAD no, nos da una pista clave: **FreeCAD es mucho más exigente con su entorno de ejecución y sus dependencias de Python que KiCAD.**

Posiblemente el problema no es de "instalación" en términos de archivos presentes, sino de **visibilidad y contexto de ejecución**. Aquí se presentan las causas más probables y cómo solucionarlas:

---

1. El binario correcto: FreeCAD vs FreeCADCmd

En un entorno de Docker (que suele ser *headless* o sin servidor gráfico), intentar ejecutar el comando freecad fallará porque este intenta levantar la interfaz de usuario (GUI) y busca un servidor X11.

- **La solución:** El Agente debe llamar a FreeCADCmd. Es la versión de consola diseñada para procesar scripts sin interfaz gráfica.
- **Prueba rápida:** Dentro del contenedor, ejecuta which FreeCADCmd. Si no devuelve nada, el binario no está en el $PATH.

2. El problema del PYTHONPATH (Causa #1)

A diferencia de KiCAD, que suele integrar su API de Python de forma más estándar en las rutas del sistema, FreeCAD instala sus módulos de Python (FreeCAD.so, Part.so, etc.) en una carpeta privada (normalmente /usr/lib/freecad/lib).

Si el Agente de IA intenta hacer un **import FreeCAD** dentro del sandbox, fallará porque el intérprete de Python no sabe dónde buscar.

- **La solución:** Debes exportar la ruta antes de ejecutar el script en Docker:
  Script Bash:
  export PYTHONPATH=$PYTHONPATH:/usr/lib/freecad/lib

  Nota: La ruta exacta depende de la distro (Debian/Ubuntu/Alpine) que se use en el Docker.

3. Dependencias de Librerías Dinámicas (ldd)

FreeCAD depende de una enorme cantidad de librerías de **Qt** y **Coin3D**. A veces, el comando aparece como "no encontrado" no porque falte el ejecutable, sino porque el cargador de enlaces dinámicos no puede encontrar una de sus dependencias (.so).

- **La solución:** Ejecuta esto dentro de tu contenedor:
  Script Bash:
  ldd $(which FreeCADCmd) | grep "not found"

  Si se ve alguna línea, falta instalar un paquete de sistema (como libtk8.6 o libpyside2) dentro del Dockerfile.

4. La trampa del AppImage

Si se instaló FreeCAD usando un **AppImage** dentro del Docker (una práctica común para tener la última versión), el comando no funcionará directamente porque requiere FUSE, y Docker no tiene privilegios de FUSE por defecto.

- **La solución:** Se debe extraer el AppImage:
  Script Bash:
  ./FreeCAD.AppImage --appimage-extract

  Y luego apuntar el Agente al ejecutable que queda dentro de la carpeta squashfs-root.

Análisis de la estructura de tu Agente

Para que el Agente procese el comando **/freecad un cilindro de radio 10 y altura 30**, el flujo de ejecución en Python dentro del Docker debería verse similar a esto para evitar errores de entorno:

Script Python:

```
import sys

# Indispensable para que Python "encuentre" FreeCAD
FREECADPATH = '/usr/lib/freecad/lib'
if FREECADPATH not in sys.path:
    sys.path.append(FREECADPATH)

try:
    import FreeCAD
    import Part
    # Lógica para crear el cilindro...
    doc = FreeCAD.newDocument("CilindroBot")
    cylinder = Part.makeCylinder(10, 30)
    Part.show(cylinder)
    doc.saveAs("resultado.FCStd")
except ImportError as e:
    print(f"Error: No se pudo cargar el módulo de FreeCAD. {e}")
```

¿Qué se sugiere revisar primero?

1. **Verifica la ruta de importación:** Entra al contenedor Docker manualmente (docker exec -it <id_contenedor> bash) e intentar ejecutar python3 -c "import FreeCAD; print('Listo')". Si falla, es el **PYTHONPATH**.
2. **Uso de Memoria:** A futuro se quiere manejar diseños de hardware (como el SoC). Entonces , hay que tener en cuenta que FreeCAD consume mucha más RAM que KiCAD al procesar geometrías. Hay que asegurarse entonces de que el contenedor Docker no tenga límites de memoria muy estrictos.

En función de esto, a continuación se propone un Dockerfile optimizado para que el Agente de IA pueda usar FreeCAD y KiCAD simultáneamente sin conflictos de librerías:

Para que el Agente de IA sea un "maestro constructor" que maneje tanto el diseño de PCBs (KiCAD) como el modelado mecánico (FreeCAD), necesitamos un Dockerfile que resuelva el conflicto de las librerías compartidas y configure el entorno Headless (sin monitor).

Aquí va la propuesta técnica de alto nivel para tu Dockerfile, optimizada para ser ligera pero totalmente funcional.

Dockerfile Optimizado: FreeCAD + KiCAD + Python Agent

Este diseño utiliza Ubuntu 22.04 como base por su estabilidad con las librerías de CAD y configura un entorno virtual para que tu Agente no rompa las dependencias del sistema.

Dockerfile

```
# 1. Base estable con soporte para Python y herramientas de sistema
FROM ubuntu:22.04

# Evitar prompts interactivos durante la instalación
ENV DEBIAN_FRONTEND=noninteractive

# 2. Instalar dependencias esenciales y repositorios de CAD
RUN apt-get update && apt-get install -y \
   software-properties-common \
   xvfb \
   libgl1-mesa-dri \
   libgl1-mesa-glx \
   python3-pip \
   python3-venv \
   wget \
   && add-apt-repository ppa:kicad/kicad-8.0-releases -y \
   && add-apt-repository ppa:freecad-maintainers/freecad-stable -y \
   && apt-get update

# 3. Instalar FreeCAD y KiCAD (Versiones de consola/librerías)
# Instalamos también las librerías de Python específicas de cada uno
RUN apt-get install -y \
   freecad-python3 \
   kicad \
   python3-kicad \
   && apt-get clean && rm -rf /var/lib/apt/lists/*

# 4. Configuración del entorno del Agente (Conda-like o Venv)
WORKDIR /app
COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

# 5. VARIABLES DE ENTORNO CRÍTICAS
# Indicamos a Python dónde encontrar los módulos de FreeCAD
ENV PYTHONPATH="/usr/lib/freecad/lib:/usr/lib/freecad-python3/lib:${PYTHONPATH}"
# Configuramos un display virtual para evitar errores de Qt/X11
ENV DISPLAY=:99
```

```
# 6. Script de inicio para levantar Xvfb y el Agente
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

ENTRYPOINT ["/entrypoint.sh"]
```

El "Truco Maestro": El archivo entrypoint.sh

FreeCAD, aunque se use FreeCADCmd, a veces intenta inicializar módulos de interfaz. Para evitar que el proceso "explote" al no encontrar un monitor, usamos Xvfb (X Virtual FrameBuffer).

Se crea un archivo llamado entrypoint.sh en la misma carpeta:

Script Bash:

```
#!/bin/bash
# Iniciar el servidor gráfico virtual en segundo plano
Xvfb :99 -screen 0 1024x768x16 &

# Ejecutar el agente de IA (el script de Python)
python execution/run_agent.py
```

¿Por qué esto soluciona el problema?

1. **Mapeo de Rutas (PYTHONPATH): El error que se tenía ocurría porque FreeCAD no se instala en las rutas estándar de Python. Al agregar /usr/lib/freecad/lib, el comando import FreeCAD en el script funcionará instantáneamente.**
2. **Aislamiento de Versiones: Al usar el PPA oficial, nos aseguramos de que las librerías de KiCAD y FreeCAD no entren en conflicto con las versiones de Python que el Agente usa para Telegram.**
3. **A futuro para la compatibilidad con el SoC: Como se quiere trabajar en el rediseño del SoC con 8MB de PSRAM extra, este contenedor permitirá:**
    - **Usar KiCAD para generar los archivos G-Code o un diseño de las 4 capas por ejemplo.**
    - **Usar FreeCAD para calcular el volumen térmico o diseñar una carcasa que soporte el chip de memoria C2766524.**

Pasos finales sugeridos:

- **En el código Python: Asegúrate de que el Agente use FreeCADCmd si se va a ejecutar scripts externos, o simplemente importa los módulos si se va a manipular**

la geometría directamente.

- Memoria: Dado que FreeCAD es intensivo, al lanzar el contenedor Docker, hay que asegurarse de darle suficiente "aire":
  docker run -m 2g --cpus 2 … (lo de los dos CPUs aun no se puede)

Escribir la función específica en Python para que el Agente traduzca "/freecad cilindro" en un script que FreeCAD pueda entender y renderizar.

Vamos a construir el "puente" lógico. El reto aquí es que el Agente IA debe actuar como un intérprete de lenguaje natural y convertir ese texto de Telegram en código que la API de FreeCAD entienda.

Dado que estamos en un entorno profesional, no usaremos simples if/else. Utilizaremos Expresiones Regulares (Regex) para extraer los parámetros y la librería Part de FreeCAD para la generación geométrica.

Estructura del Procesador de Comandos

Este script viviría dentro del Agente en el contenedor Docker. Se ha diseñado la función para que sea extensible (se pueden añadir cubos, esferas o incluso exportar a STL automáticamente).

Script Python

```
import re
import os

# Intentar importar FreeCAD (asegúrate que el PYTHONPATH esté configurado)
try:
    import FreeCAD as App
    import Part
except ImportError:
    print("Error: Los módulos de FreeCAD no están en el path.")

def procesar_comando_freecad(mensaje_telegram):
    """
    Analiza: "/freecad un cilindro de radio 10 y altura 30"
    Retorna: Ruta del archivo generado o mensaje de error.
    """
    # 1. Limpieza y normalización del mensaje
    texto = mensaje_telegram.lower().replace("/freecad", "").strip()

    # 2. Definición de patrones (Regex)
    # Buscamos 'cilindro', un número para radio y un número para altura
```

```
    patron_cilindro = r"cilindro.*radio\s*(\d+\.?\d*).*altura\s*(\d+\.?\d*)"

    match = re.search(patron_cilindro, texto)

    if match:
        radio = float(match.group(1))
        altura = float(match.group(2))

        return generar_cilindro(radio, altura)
    else:
        return "No entendí los parámetros. Prueba con: 'cilindro de radio X y altura Y'"

def generar_cilindro(r, h):
    """Lógica pura de FreeCAD"""
    doc_name = "PiezaGenerada"
    archivo_salida = f"/tmp/{doc_name}.step" # Formato estándar industrial

    try:
        # Crear documento nuevo
        doc = App.newDocument(doc_name)

        # Crear el cilindro
        cilindro = Part.makeCylinder(r, h)

        # Añadirlo al documento como un objeto
        objeto_en_doc = doc.addObject("Part::Feature", "CilindroIA")
        objeto_en_doc.Shape = cilindro

        # Recomputar geometría
        doc.recompute()

        # Exportar a STEP (ideal para seguir trabajando en el diseño del SoC)
        Part.export([objeto_en_doc], archivo_salida)

        # Opcional: Guardar el proyecto nativo .FCStd
        # doc.saveAs(f"/tmp/{doc_name}.FCStd")

        App.closeDocument(doc_name)
        return archivo_salida

    except Exception as e:
        return f"Error técnico en FreeCAD: {str(e)}"
```

```
# --- Ejemplo de uso por el Agente ---
# user_input = "/freecad un cilindro de radio 5.5 y altura 20"
# resultado = procesar_comando_freecad(user_input)
# if resultado.endswith(".step"):
#     enviar_archivo_a_telegram(resultado)
```

¿Por qué este enfoque es el correcto?

1. **Flexibilidad de Formato: Se ha configurado la salida en .STEP. Como se quiere para el desarrollo de un SoC con 8MB de PSRAM extra, el formato STEP es el que se podría importar luego en KiCAD (usando el *Footprint Editor*) para verificar que la carcasa mecánica no choque con los componentes del PCB.**
2. **Manejo de Flotantes: El Regex (\d+\.?\d*) permite que el usuario pida radios con decimales (ej. 5.5mm), lo cual es vital en ingeniería de precisión.**
3. **Gestión de Memoria: Al usar App.closeDocument(), se evita que el contenedor Docker se sature de RAM si el Agente recibe muchas peticiones seguidas de Telegram.**

Integración con el flujo de Telegram

El Agente de IA (en el main_agent.py) debería tener un bloque similar a este para cerrar el ciclo:

- **Paso A: El Agente detecta el comando /freecad.**
- **Paso B: Llama a procesar_comando_freecad().**
- **Paso C: Si recibe una ruta de archivo, usa la API de Telegram (send_document) para devolverle al usuario la pieza en 3D lista para imprimir o integrar.**

Una función para que el Agente IA también genere automáticamente una captura de pantalla (PNG) de la pieza para que el usuario pueda verla en Telegram antes de descargar el archivo.

Ver la pieza directamente en el chat de Telegram antes de descargar el archivo .STEP mejora muchísimo la experiencia de usuario (UX).

Para lograr esto en un entorno *headless* (sin monitor), usaremos el Xvfb que configuramos en el entrypoint.sh anterior. Esto permite que FreeCAD "crea" que tiene una pantalla donde dibujar la pieza y así poder tomar la "foto".

Función de Renderizado y Captura (PNG)

Añadiremos el módulo FreeCADGui al script. Este código debe ir dentro del mismo bloque donde se genera la pieza:

Script Python:

```
try:
    import FreeCAD as App
    import FreeCADGui as Gui  # Necesario para el renderizado
    import Part
except ImportError:
    print("Error: Librerías de GUI no encontradas.")

def capturar_vista_3d(ruta_png):
    """
    Configura la cámara y toma una captura de pantalla de la pieza actual.
    """
    try:
        # 1. Inicializar la vista de la GUI (en el display virtual :99)
        Gui.setupGui()
        view = Gui.activeDocument().activeView()

        # 2. Configurar una vista isométrica estándar
        view.viewIsometric()
        view.fitAll() # Centrar la pieza en "cámara"

        # 3. Guardar la imagen (fondo transparente o color sólido)
        # Parámetros: (ruta, ancho, alto, color_fondo)
        view.saveImage(ruta_png, 800, 600, (0.3, 0.3, 0.3))

        return ruta_png
    except Exception as e:
        return f"Error en renderizado: {str(e)}"
```

Integración en el Flujo del Agente

Ahora, actualizamos la función generar_cilindro para que devuelva ambos archivos: el modelo 3D y la previsualización.

Python

```
def generar_cilindro_con_preview(r, h):
    doc_name = "PiezaAgente"
    archivo_step = f"/tmp/{doc_name}.step"
    archivo_png = f"/tmp/{doc_name}.png"

    try:
        doc = App.newDocument(doc_name)
```

```
        cilindro = Part.makeCylinder(r, h)
        obj = doc.addObject("Part::Feature", "Cilindro")
        obj.Shape = cilindro
        doc.recompute()

        # Guardar el STEP para ingeniería
        Part.export([obj], archivo_step)

        # Generar el PNG para el usuario de Telegram
        capturar_vista_3d(archivo_png)

        App.closeDocument(doc_name)
        return archivo_step, archivo_png

    except Exception as e:
        return None, str(e)
```

¿Por qué esto es útil para el futuro proyecto de SoC?

Como se quiere trabajar en el diseño de un SoC con 8MB de PSRAM extra (C2766524), esta función permite hacer lo siguiente por Telegram:

1. **Validación Visual Rápida: Si se le pide al Agente IA que diseñe un "disipador" o una "carcasa" para ese chip específico, se verá el PNG al instante para confirmar si las proporciones son correctas antes de pasarlo a KiCAD.**
2. **Ahorro de Datos: No se necesita descargar el archivo pesado si se ve en la imagen que el Agente IA cometió un error en el radio o la altura.**
3. **Documentación: Se puedes reenviar esas capturas a mis colegas de “Tecnología Venezolana” que poseen una CNC para validar avances mecánicos.**

Notas técnicas para el Agente de IA:

- **Xvfb: Hay que asegurarse de que el comando `Xvfb :99` esté corriendo antes de llamar a `Gui.setupGui()`, de lo contrario, el Agente lanzará un error de *Segmentation Fault*.**
- **Limpieza: Hay que recuerdar programar al Agente IA para que borre los archivos de `/tmp/` después de enviarlos a Telegram para no llenar el almacenamiento del contenedor.**

Comando para que el Agente pueda "unir" el diseño de KiCAD con el de FreeCAD, por ejemplo, para verificar colisiones entre el PCB y la carcasa.

Esto es llevar el Agente de IA al siguiente nivel: el diseño asistido por computadora

(CAD) con validación mecánica automática.

Para lograr esto, el Agente debe realizar un "Assembly" (ensamblaje) virtual. El flujo técnico será:

1. Exportar el PCB desde KiCAD a un formato que FreeCAD entienda (STEP).
2. Importar ambos modelos en el entorno de FreeCAD.
3. Ejecutar un análisis de interferencia (boolean intersection).

Aquí se tiene cómo implementar esta función crítica:

1. Exportación de KiCAD a STEP (Lado Electrónico)

KiCAD tiene una herramienta de línea de comandos llamada kicad-cli (en la versión 8.0+ que pusimos en el Dockerfile) que es perfecta para esto:

Script Python:

```
import subprocess

def exportar_kicad_a_step(archivo_pcb, carpeta_salida):
    """
    Convierte el diseño de 4 capas del SoC en un modelo 3D.
    """
    nombre_step = archivo_pcb.replace(".kicad_pcb", ".step")
    try:
        # Comando oficial de KiCAD 8.0 para exportar 3D sin abrir la GUI
        subprocess.run([
            "kicad-cli", "pcb", "export", "step",
            "--output", nombre_step,
            archivo_pcb
        ], check=True)
        return nombre_step
    except Exception as e:
        return f"Error exportando KiCAD: {str(e)}"
```

2. Verificación de Colisiones en FreeCAD (Lado Mecánico)

Ahora, el Agente usará la lógica de "Intersección". Si dos objetos ocupan el mismo espacio, la intersección tendrá un volumen mayor a cero.

Script Python:

```
def verificar_ajuste_mecanico(ruta_step_pcb, ruta_step_carcasa):
    """
    Importa ambos y busca colisiones.
    """
```

```
    try:
        doc = App.newDocument("Validacion")

        # Importar el PCB (con el chip PSRAM C2766524 y las 4 capas)
        pcb_part = Part.read(ruta_step_pcb)
        obj_pcb = doc.addObject("Part::Feature", "PCB")
        obj_pcb.Shape = pcb_part

        # Importar la carcasa diseñada por el Agente
        carcasa_part = Part.read(ruta_step_carcasa)
        obj_carcasa = doc.addObject("Part::Feature", "Carcasa")
        obj_carcasa.Shape = carcasa_part

        # OPERACIÓN CRÍTICA: Intersección (Common)
        # Si hay colisión, este objeto tendrá volumen
        interseccion = obj_pcb.Shape.common(obj_carcasa.Shape)
        volumen_colision = interseccion.Volume

        resultado = ""
        if volumen_colision > 0.1: # Tolerancia de 0.1mm3
            resultado = f"⚠️ COLISIÓN DETECTADA: Hay una interferencia de
{round(volumen_colision, 2)} mm³."
            # Generar imagen de la colisión para mostrar dónde falla
            obj_pcb.ViewObject.Transparency = 50 # Hacer el PCB semitransparente
            capturar_vista_3d("/tmp/error_ajuste.png")
        else:
            resultado = "✅ AJUSTE PERFECTO: El PCB encaja correctamente en la carcasa."

        App.closeDocument("Validacion")
        return resultado

    except Exception as e:
        return f"Error en la validación: {str(e)}"
```

3. El comando de Telegram: /validar

El Agente IA ahora puede responder a algo como: ***"/validar mi SoC con la carcasa actual"***

Lo que el Agente hará internamente:

1. **Busca el último archivo .kicad_pcb donde se añadió la PSRAM C2766524.**
2. **Lo convierte a 3D.**
3. **Lo mete dentro del cilindro/caja que se diseñó antes.**

4. Responde con el texto de validación y una imagen.

¿Por qué esto es vital para el diseño de 4 capas para el SoC?

Al pasar de 2 a 4 capas y añadir la memoria de alta velocidad, el grosor del PCB o la altura de los componentes (como los capacitores de desacoplo C14663) podrían chocar con una carcasa muy ajustada.

Ventajas de este flujo:

- Impedancia Controlada: Al no tener que mover componentes manualmente para que "quepan", se mantiene la integridad de las pistas de 50 ohms que se definió.
- Ahorro de Prototipos: Se Sabrá si el conector o la memoria chocan antes de enviar a fabricar a la JLC o al proveedor habitual.

¿Qué necesitas hacer ahora?

Asegúrate de que en el Dockerfile, la versión de KiCAD sea la 8.0+, ya que las anteriores no incluían el kicad-cli de forma tan robusta para exportaciones automáticas.

Manejador de mensajes del Agente IA para que reconozca los archivos adjuntos que le se le envíen por Telegram.

Para cerrar este ecosistema de ingeniería, necesitamos que el Agente IA no solo ejecute comandos, sino que sea capaz de recibir archivos, guardarlos en el sandbox y procesarlos.

Se usará la librería python-telegram-bot (una de las más robustas), pero la lógica es adaptable a cualquier otra.

1. El Manejador de Archivos y Comandos

Este código permite al Agente "recordar" cuál es el último PCB y la última carcasa que ha manejado para poder validarlos.

Script Python:

```
import os
from telegram import Update
from telegram.ext import Application, CommandHandler, MessageHandler, filters,
ContextTypes

# Directorios de trabajo dentro del Docker
UPLOAD_DIR = "/app/workdir"
os.makedirs(UPLOAD_DIR, exist_ok=True)
```

```
# Variables de estado (en un entorno real, usarías una pequeña DB o JSON)
last_files = {
    "pcb": None,
    "mecanico": None
}

async def guardar_documento(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """
    Recibe archivos .kicad_pcb o .step y los organiza.
    """
    archivo = await update.message.document.get_file()
    nombre_original = update.message.document.file_name
    ruta_destino = os.path.join(UPLOAD_DIR, nombre_original)

    await archivo.download_to_drive(ruta_destino)

    if nombre_original.endswith(".kicad_pcb"):
        last_files["pcb"] = ruta_destino
        await update.message.reply_text(f"✅ PCB recibido: {nombre_original}. Listo para
validar con la carcasa.")

    elif nombre_original.endswith((".step", ".stp", ".FCStd")):
        last_files["mecanico"] = ruta_destino
        await update.message.reply_text(f"✅ Modelo mecánico recibido:
{nombre_original}.")

async def comando_validar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """
    Comando /validar: Cruza el PCB con la carcasa.
    """
    if not last_files["pcb"] or not last_files["mecanico"]:
        await update.message.reply_text("❌ Error: Necesito que me envíes primero el
archivo .kicad_pcb y el modelo 3D.")
        return

    await update.message.reply_text("🔍 Iniciando validación de colisiones (DRC
Mecánico)...")

    # 1. Exportar KiCAD a STEP (Usando la función que vimos antes)
    ruta_step_pcb = exportar_kicad_a_step(last_files["pcb"], UPLOAD_DIR)

    # 2. Verificar colisiones en FreeCAD
    resultado_texto = verificar_ajuste_mecanico(ruta_step_pcb, last_files["mecanico"])
```

```
    # 3. Enviar respuesta con la imagen si hubo error
    await update.message.reply_text(resultado_texto)

    if "COLISIÓN" in resultado_texto:
        if os.path.exists("/tmp/error_ajuste.png"):
            with open("/tmp/error_ajuste.png", "rb") as photo:
                await update.message.reply_photo(photo, caption="Puntos de interferencia
detectados.")

# --- Configuración del Bot ---
if __name__ == '__main__':
    # Reemplaza con tu Token de Telegram
    application = Application.builder().token("TU_TELEGRAM_TOKEN").build()

    # Manejar documentos
    application.add_handler(MessageHandler(filters.Document.ALL, guardar_documento))

    # Manejar el comando de validación
    application.add_handler(CommandHandler("validar", comando_validar))

    # Manejar el comando de FreeCAD que hicimos antes
    # application.add_handler(CommandHandler("freecad", comando_freecad_directo))

    print("Agente de Ingeniería iniciado y esperando archivos...")
    application.run_polling()
```

¿Cómo cambia esto tu flujo de trabajo con el SoC?

Ahora tu flujo de diseño con la PSRAM C2766524 sería así de eficiente:

1. **En el PC de escritorio: Terminas de trazar las pistas de 50 ohms en el PCB de 4 capas en KiCAD.**
2. **En Telegram: Arrastras el archivo .kicad_pcb al chat con el Agente IA.**
3. **El Agente: Responde "PCB recibido".**
4. **En Telegram: Se escribe /freecad un cilindro de radio 25 y altura 10 (para crear una carcasa rápida).**
5. **En Telegram: Se escribe /validar.**
6. **El Agente: Dirá si el chip de memoria o los conectores chocan con la tapa de ese cilindro y enviará una foto del "rayo X" de la interferencia.**

Recomendación final:

Como se está usando Conda, hay que asegúrarse de que el entorno de Conda dentro del Docker tenga instalada la librería python-telegram-bot.

Si en el futuro se decide usar el script de G-Code Sender que se mencionó antes, se podría añadir un comando /fabricar que envíe el archivo validado directamente a la máquina CNC o impresora 3D.